

Operation Steps:

Step 1 - Turn ON the Relay 1(R1)

Step 2 - After 30Sec of Turn ON Relay 1(R1) then Main Relay1(MR1) will ON.

Step 3 - When the floats sensor get triggers then cutoff the Main Realy1(MR1) then after 30Sec, cutoff the Relay1(R1).

Step 4 - Then Rest For 1Min

Step 5 - After 1Min Rest Delay, Turn ON the Relay2(R2).

Step 6 - After 30Sec of Turn ON Relay2(R2) then Main Relay1(MR1) will ON.

Step 7 - When the floats sensor get triggers then cutoff the Main Realy1(MR1) then after 30Sec, cutoff the Relay2(R2).

Step 8 - Again Rest For 1Min.

Step 9 - After 1Min Rest Delay, Turn ON the Relay3(R3).

Step 10 - After 30Sec of Turn ON Relay3(R3) then Main Relay2(MR2) will ON.

Step 11 - When the floats sensor get triggers then cutoff the Main Realy2(MR2) then after 30Sec, cutoff the Relay3(R3).

Step 12 - Again Rest For 1Min.

Step 13 - After 1Min Rest Delay, Turn ON the Relay4(R4).

Step 14 - After 30Sec of Turn ON Relay4(R4) then Main Relay2(MR2) will ON.

Step 15 - When the floats sensor get triggers then cutoff the Main Realy2(MR2) then after 30Sec, cutoff the Relay4(R4).

Step 16 - Again Rest For 1Min.

Step 17 - After 1Min Rest Delay, Turn ON the Relay5(R5).

Step 18 - After 30Sec of Turn ON Relay5(R5) then Main Relay2(MR2) will ON.

Step 19 - When the floats sensor get triggers then cutoff the Main Realy2(MR2) then after 30Sec, cutoff the Relay5(R5).

Step 20 - Again Rest For 1Min.

And I Want to Seperate the Process like Functions :

Functions:

Function 1 - it contains Relay1(R1), Relay2(R2) And Main Realy1(MR1) (Step 1 to Step 8).

Function 2 - It Contains Relay3(R3), Relay4(R4), Relay5(R5) And Main Relay2(MR2) (Step 9 to Step 20).

And I Want to Add the I2C LCD Display(16x2) and also RTC Module For Scheduling and Add Bluetooth to control function and set delay time and control the operation and functions.

Bluetooth Interface:

Want Bluetooth interface for control the system in the mobile like operation control, Function Control, Time delay Control.

Need Bluetooth Response Text for this and need to work operation when I enter the cmd like given below.

CMD For LLS Test Bench:

- | | |
|---------------|---|
| 1). STARTF1 | - Start the Function 1 |
| 2). STOPF1 | - Stop the Function 1 |
| 3). STARTF2 | - Start the Function 2 |
| 4). STOPF2 | - Stop the Function 2 |
| 5). STARTBOTH | - Start the Function 1 And Function 2 |
| 6). STOPBOTH | - Stop the Function 1 And Function 2 |
| 7). T2 | - Time Delay(Rest) Set for 2Mins (For that 1Mins Change Into 2Mins) |
| 8). DAY01OFF | - Shutdown the System or Stop the Process(Operations) for Sunday |
| 9). DAY02OFF | - Shutdown the System or Stop the Process(Operations) for Monday |
| 10). DAY03OFF | - Shutdown the System or Stop the Process(Operations) for Tuesday |
| 11). DAY04OFF | - Shutdown the System or Stop the Process(Operations) for Wednesday |
| 12). DAY05OFF | - Shutdown the System or Stop the Process(Operations) for Thursday |
| 13). DAY06OFF | - Shutdown the System or Stop the Process(Operations) for Friday |
| 14). DAY07OFF | - Shutdown the System or Stop the Process(Operations) for Saturday |

- 15). DAY01ON - Start the System and the Process for Sunday
- 16). DAY02ON - Start the System and the Process for Monday
- 17). DAY03ON - Start the System and the Process for Tuesday
- 18). DAY04ON - Start the System and the Process for Wednesday
- 19). DAY05ON - Start the System and the Process for Thursday
- 20). DAY06ON - Start the System and the Process for Friday
- 21). DAY07ON - Start the System and the Process for Saturday
- 22). RESETMEM - Reset the EEPROM Memory
- 23). SETDATE=YYYY-MM-DD – For Date setting in RTC
- 24). LOG - Check Log Status of the Device
- 25). CMDS / HELP / ? - Find or list out all cmds
- 26). HIST - For Check History logs
- 27). SETDOW=xx - For Set Day of Week (01=Sunday)
- 28). SETTIME=HH:MM:SS – For Set Time in RTC
- 29). RTx - For Set time delay Between Relay and motor relay.

RTC Module (DS3231) Interface:

Add RTC Module For RealTime Scheduling the Process(Operation) and Function and check the realtime Log status.

I2C LCD Display(16x2) Interface:

Want to Add the I2C LCD Display for Monitoring the Functions And Operation(Process)

And Print(Display) the Realys Status like Given Below,

- 1). When Relay(R1) is ON then Print "T1 Fill/T2 Drain".
- 2). When Relay(R2) is ON then Print "T2 Fill/T1 Drain".
- 3). When Relay(R3) is ON then Print "T3 Fill/T5 Drain".
- 4). When Relay(R4) is ON then Print "T4 Fill/T3 Drain".

5). When Relay(R5) is ON then Print "T5 Fill/T4 Drain".

6). When Main Realy1(MR1) is ON then Print "Motor 1 ON".

7). When Main Realy2(MR2) is ON then Print "Motor 2 ON".

8). When Main Realy1(MR1) is OFF then Print "Motor 1 OFF".

9). When Main Realy2(MR2) is OFF then Print "Motor 2 OFF".

10). When Both Main Realy1(MR1) & Main Realy2(MR2) is ON then Print "Motor 1 & 2 ON"

11). When Both Main Realy1(MR1) & Main Realy2(MR2) is OFF then Print "Motor 1 & 2 OFF"

print which realy is on even 2or3 relays also in ON state print that in same line in the display

and also print the main relays in line 2 of the display

and also it will Automatically scroll to Next.

PIC pin #	MCU pin	Dir	Used for	External connection / notes
1	MCLR/VPP/RE3	—	Reset/ICSP	External MCLR enabled (config MCLRE=EXTMCLR). Pull-up ~10 k Ω to VDD; keep ICSP header clear.
2	RA0	OUT	MR1 (Motor 1 relay)	Drives relay coil (through driver/transistor).
3	RA1	OUT	MR2 (Motor 2 relay)	Drives relay coil (through driver/transistor).
4	RA2	IN	FS5 (float 5, active-LOW)	Needs external pull-up ($\approx$ 10 k Ω to VDD). Switch to GND when “ON”.
5	RA3	—	Spare	Digital I/O available (ANSELA=0 disables analog).
6	RA4	—	Spare	Digital I/O available.
7	RA5	—	Spare	Digital I/O available.
8	VSS	—	Ground	0 V. Tie to system GND.

PIC pin #	MCU pin	Dir	Used for	External connection / notes
9	RA7	—	Spare	Digital I/O available.
10	RA6	—	Spare	Digital I/O available.
11	RC0	IN	FS1 (float 1, active-LOW)	Needs external pull-up ($\approx 10\text{ k}\Omega$ to VDD).
12	RC1	IN	FS2 (float 2, active-LOW)	Needs external pull-up ($\approx 10\text{ k}\Omega$ to VDD).
13	RC2	IN	FS3 (float 3, active-LOW)	Needs external pull-up ($\approx 10\text{ k}\Omega$ to VDD).
14	RC3/SCL1	I ² C	I²C SCL	Shared bus: DS3231 RTC (0x68) + LCD PCF8574 (0x27). 4.7 k Ω pull-up to VDD.
15	RC4/SDA1	I ² C	I²C SDA	Shared bus. 4.7 k Ω pull-up to VDD.
16	RC5	—	Spare	Digital I/O available.
17	RC6/TX1	OUT	UART TX (to HC-05 RXD)	9600 bps. HC-05 RX is 3.3 V tolerant—use divider if you want to be safe.
18	RC7/RX1	IN	UART RX (from HC-05 TXD)	9600 bps.
19	VSS	—	Ground	0 V. Tie to system GND.
20	VDD	—	+5 V	Decouple ($\geq 0.1\text{ }\mu\text{F}$ near VDD/VSS).
21	RB0	OUT	R1 relay	Tank/valve relay output (driver required).
22	RB1	OUT	R2 relay	Relay output.
23	RB2	OUT	R3 relay	Relay output.
24	RB3	OUT	R4 relay	Relay output.
25	RB4	OUT	R5 relay	Relay output.
26	RB5	IN	FS4 (float 4, active-LOW)	Uses internal pull-up (code sets <code>INTCON2.RBPU=0, WPUB5=1</code>). Add external if needed.
27	RB6/PGC	—	ICSPCLK / Spare	Keep series resistor if used externally (don't load heavily).
28	RB7/PGD	—	ICSPDAT / Spare	Same note as RB6.

/* LLS_Test_Bench_Auto.c ? PIC18F26K22 ? LLS Test Bench (HC-05 + DS3231 + I2C 16x2 LCD)

*

* This build:

* - Graceful STOP when DAYxxOFF or midnight rolls into an OFF day (post-delay & rest).

* - UART prints LF-only (no CR) -> no '^M'.

* - OFF-day message blocks START* on an OFF day (no auto-resume on DAYxxON).

* - LOG: prints one up-to-date snapshot only (no spam). HIST keeps history.

* - LOG now also shows RT(pre), RT(post) and T(rest) times (MM:SS).

* - RAM log de-duplicates identical snapshots on write (not only on dump).

* - RTC shadow persisted to EEPROM and used if DS3231 OSF is set.

* - Resume from EEPROM snapshot after power loss.

* - LCD idle: banner + live date/day/time; busy: relay text + motor line, auto-scroll.

* - Cmds: STARTF1/STOPF1, STARTF2/STOPF2, STARTBOTH/STOPBOTH,

* DAY01ON/OFF..DAY07ON/OFF, RTC?/RTC/TIME?/TIME,

* SETTIME=HH:MM[:SS], SETDATE=YYYY-MM-DD,

* SETDATETIME=YYYY-MM-DD,HH:MM:SS, SETDOW=01..07,

* LOG, HIST, CLEARLOG, QUIET, VERBOSE, HELP/CMDS/?,

* RTxx (set both 30s pre/post delays to xx minutes; e.g., RT1 = 1 minute),

* Txx (set rest time to xx minutes; e.g., T60 = 60 minutes),

* DTxx (set single-drain time in seconds; e.g., DT30 = 30s, DT120 = 120s),

* DRAIN1..DRAIN5 (single-drain by tank index; LCD shows "Draining Process" + "Tank N").

*

* - NEW LCD during Function 1/2 (when running and a float is HIGH, debounced):

* Float 1 High -> "Tank 1 Full - Tank 2 Low"

* Float 2 High -> "Tank 2 Full - Tank 1 Low"

* Float 3 High -> "Tank 3 Full - Tank 4 & 5 Low"

* Float 4 High -> "Tank 4 Full - Tank 3 & 5 Low"

* Float 5 High -> "Tank 5 Full - Tank 4 & 5 Low"

*

- * Robustness fixes retained:
- * - High-priority TMRO & EUSART RX (TMR0IP=1, RC1IP=1), OERR/FERR handling.
- * - TRISC6=0 (TX out), TRISC7=1 (RX in).
- * - Atomic 32-bit millis() and EEPROM write critical section (both priorities locked).
- * - START after STOP always works (force-clean start semantics).
- * - I2C Standard-mode timing (SMP=1).
- */

```
#include <xc.h>
```

```
#include <stdint.h>
```

```
#include <stdbool.h>
```

```
#include <string.h>
```

```
#include <stdio.h>
```

```
/* ===== CONFIG BITS ===== */
```

```
#pragma config FOSC = INTIO67
```

```
#pragma config PLLCFG = OFF
```

```
#pragma config PRICLK = ON
```

```
#pragma config FCMEN = OFF
```

```
#pragma config IESO = OFF
```

```
#pragma config PWRTE = ON
```

```
#pragma config BOREN = SBORDIS
```

```
#pragma config BORV = 190
```

```
#pragma config WDTE = OFF
```

```
#pragma config WDTPS = 1024
```

```
#pragma config CCP2MX = PORTC1
```



```
#pragma config PBADEN = OFF
#pragma config CCP3MX = PORTB5
#pragma config HFOFST = ON
#pragma config T3CMX = PORTC0
#pragma config P2BMX = PORTB5
#pragma config MCLRE = EXTMCLR
```

```
#pragma config STVREN = ON
#pragma config LVP = OFF
#pragma config XINST = OFF
#pragma config DEBUG = OFF
```

```
#pragma config CP0 = OFF, CP1 = OFF, CP2 = OFF, CP3 = OFF, CPB = OFF, CPD = OFF
#pragma config WRT0 = OFF, WRT1 = OFF, WRT2 = OFF, WRT3 = OFF, WRTC = OFF, WRTB = OFF, WRTD = OFF
#pragma config EBTR0 = OFF, EBTR1 = OFF, EBTR2 = OFF, EBTR3 = OFF, EBTRB = OFF
```

```
/* ===== CLOCK ===== */
```

```
#define _XTAL_FREQ 16000000UL
```

```
/* ===== I/O MAP ===== */
```

```
// Relays
```

```
#define R1_LAT    LATBbits.LATB0
#define R2_LAT    LATBbits.LATB1
#define R3_LAT    LATBbits.LATB2
#define R4_LAT    LATBbits.LATB3
#define R5_LAT    LATBbits.LATB4
#define MR1_LAT   LATAbits.LATA0
#define MR2_LAT   LATAbits.LATA1
```

```
#define R1_TRIS  TRISBbits.TRISB0
#define R2_TRIS  TRISBbits.TRISB1
#define R3_TRIS  TRISBbits.TRISB2
#define R4_TRIS  TRISBbits.TRISB3
#define R5_TRIS  TRISBbits.TRISB4
#define MR1_TRIS TRISAbits.TRISA0
#define MR2_TRIS TRISAbits.TRISA1
```

```
// Float inputs (active-LOW)
```

```
#define FS1_PORT  PORTCbits.RC0
#define FS2_PORT  PORTCbits.RC1
#define FS3_PORT  PORTCbits.RC2
#define FS4_PORT  PORTBbits.RB5
#define FS5_PORT  PORTAbits.RA2
```

```
#define FS1_TRIS  TRISCbits.TRISC0
#define FS2_TRIS  TRISCbits.TRISC1
#define FS3_TRIS  TRISCbits.TRISC2
#define FS4_TRIS  TRISBbits.TRISB5
#define FS5_TRIS  TRISAbits.TRISA2
```

```
/* ===== CONSTANTS ===== */
```

```
#define I2C_LCD_ADDR  0x27
#define I2C_RTC_ADDR  0x68
```

```
// Timing (ms)
```

```
#define REST_DEFAULT_MS    60000UL
#define REST_MAX_MS        (12UL*60UL*60UL*1000UL) /* up to 12 hours */
#define FLOAT_DEBOUNCE_MS  120UL
```

```
// LCD scroll period
```

```
#define LCD_SCROLL_PERIOD_MS 400UL
```

```
// UART
```

```
#define UART_RX_BUF_SZ 64
```

```
// RAM log depth
```

```
#define RAM_LOG_DEPTH 16
```

```
// EEPROM settings
```

```
#define EE_SIGNATURE_ADDR 0x00
```

```
#define EE_SIGNATURE_VALUE 0x5Au
```

```
#define EE_DAYMASK_ADDR 0x01
```

```
#define EE_REST_MS_ADDR 0x02 // 4 bytes (uint32_t)
```

```
/* EEPROM for pre/post delays */
```

```
#define EE_PRE_MS_ADDR 0x06 // 4 bytes
```

```
#define EE_POST_MS_ADDR 0x0A // 4 bytes
```

```
#define PREPOST_DEFAULT_MS 30000UL
```

```
#define PREPOST_MAX_MS (10UL*60UL*1000UL) // cap at 10 minutes
```

```
/* EEPROM for single-drain duration */
```

```
#define EE_DRAIN_MS_ADDR 0x0E // 4 bytes
```

```
#define DRAIN_SINGLE_DEFAULT_MS (60UL*1000UL) // default 60s
```

```
#define DRAIN_SINGLE_MAX_MS (10UL*60UL*1000UL) // cap at 10 minutes
```

```
// EEPROM RTC shadow (persist RTC if OSF)
```

```
#define EE_RTC_SHADOW_ADDR 0x20
```

```

#define EE_RTC_MAGIC      0xB1

// EEPROM persistent ring snapshots (resume only)

#define EE_LOG_BASE      0x40
#define EE_LOG_REC_SZ    20
#define EE_LOG_SLOTS     48
#define EE_LOG_MAGIC     0xA5

/* ===== GLOBAL TIME ===== */
volatile uint32_t g_millis = 0;

/* ===== SETTINGS ===== */
static uint8_t dayMask = 0x7F;
static uint32_t restDelayMs = REST_DEFAULT_MS;

/* runtime-tunable delays */
static uint32_t preMotorDelayMs    = PREPOST_DEFAULT_MS; // pre-motor delay
static uint32_t postMotorRelOffMs  = PREPOST_DEFAULT_MS; // post-motor relay-off delay

/* per-relay single drain time (seconds via DTxx) */
static uint32_t drainSingleMs      = DRAIN_SINGLE_DEFAULT_MS;

/* ===== UART RX ===== */
static volatile char rxBuf[UART_RX_BUF_SZ];
static volatile uint8_t rxHead = 0;

/* ===== RAM LOG ===== */
typedef struct {
    uint8_t yOff, mon, day, dow;

```

```

uint8_t hour, min, sec;

uint8_t flags; // b0:F1run b1:F2run b2:MR1 b3:MR2 b4:DAYenabled

uint8_t relBits; // b0..b4 => R1..R5
} ram_rec_t;


static ram_rec_t ramLog[RAM_LOG_DEPTH];
static uint8_t ramLogHead = 0, ramLogCount = 0;


/* ===== CHATTER CONTROL ===== */
static bool btVerbose = false; // QUIET by default


/* ===== FSM ===== */
typedef enum {
    ST_IDLE = 0,
    ST_PRE_MOTOR_DELAY,
    ST_MOTOR_ON_WAIT_FLOAT,
    ST_POST_MOTOR_OFF_DELAY,
    ST_REST_DELAY
} fsm_state_t;


#define SEQ_MAX 4 /* 3 steps + 0 terminator */


typedef struct {
    fsm_state_t state;

    uint8_t sequence[SEQ_MAX]; // F1: {1,2,0,0}, F2: {3,4,5,0}
    uint8_t idx;

    uint8_t motor; // 1 => MR1, 2 => MR2
    uint32_t tMark; // state start (ms)

    bool running;

```

```

    bool stop_requested;

    bool loop;
} fsm_t;


static fsm_t F1, F2;


/* ===== SINGLE-DRAIN STATE ===== */
static bool  drain_single_active = false;
static uint8_t drain_single_relay = 0; /* 1..5 */
static uint32_t drain_single_mark = 0;


/* ===== FWD DECLS ===== */
static void init_clock(void);
static void init_gpio(void);
static void init_timer0(void);
static void init_uart(void);
static void init_i2c(void);


static void uart_putc(char c);
static void uart_puts(const char *s);
static bool uart_line_available(char *outLine, uint8_t maxLen);


/* helpers */
static void print_help(void);
static void print_status_now(void);
static void fmt_mmss(uint32_t ms, char *out, uint8_t outsz);


static void i2c_start(void);
static void i2c_stop(void);

```

```

static void i2c_restart(void);

static bool i2c_write(uint8_t b);

static uint8_t i2c_read(bool ack);


/* LCD */

static void lcd_init(void);

static void lcd_clear(void);

static void lcd_set_cursor(uint8_t row, uint8_t col);

static void lcd_print(const char *s);

static void lcd_task(void);


/* RTC + helpers */

static uint8_t bcd2bin(uint8_t b);

static uint8_t bin2bcd(uint8_t b);


static uint8_t rtc_read_reg(uint8_t reg);

static void rtc_write_reg(uint8_t reg, uint8_t val);

static void rtc_sanitize(void);

static void rtc_read(uint8_t *sec, uint8_t *min, uint8_t *hour, uint8_t *dow);

static void rtc_read_full(uint8_t *sec, uint8_t *min, uint8_t *hour, uint8_t *dow,
                          uint8_t *date, uint8_t *month, uint16_t *year);

static void rtc_write_full(uint8_t sec, uint8_t min, uint8_t hour, uint8_t dow,
                          uint8_t date, uint8_t month, uint16_t year);

static uint8_t dow_from_ymd(uint16_t y, uint8_t m, uint8_t d);

static const char* dow_str(uint8_t dow);


/* EEPROM */

static void eeprom_write_byte(uint16_t addr, uint8_t val);

static uint8_t eeprom_read_byte(uint16_t addr);

```

```
static void eeprom_write_block(uint16_t addr, const uint8_t *buf, uint8_t len);

static void eeprom_read_block(uint16_t addr, uint8_t *buf, uint8_t len);

static void load_settings(void);

static void save_settings(void);


/* RTC shadow */

static void rtc_shadow_save(uint16_t yr, uint8_t mo, uint8_t dt, uint8_t dow, uint8_t hh, uint8_t mm,
uint8_t ss);

static bool rtc_shadow_load(uint16_t *yr, uint8_t *mo, uint8_t *dt, uint8_t *dow, uint8_t *hh, uint8_t
*mm, uint8_t *ss);


/* Control */

static void relays_all_off(void);

static void relays_function_off(uint8_t fn);

static void motor_set(uint8_t motor, bool on);

static void relay_set(uint8_t relay, bool on);

static bool relay_get(uint8_t relay);

static bool float_triggered(uint8_t relay);


/* NEW: debounced float read for LCD display */

static bool float_active_stable(uint8_t idx);


static bool today_enabled(void);

static bool dow_enabled(uint8_t dow);

static void parse_command(const char *cmd);

static void log_event(const char *msg);

static void ram_log_dump(void);

static void ram_log_print_latest(void);
```



```

/* FSM ops */

static void fsm_start(fsm_t *F);

static void fsm_request_stop(fsm_t *F);

static void fsm_abort(fsm_t *F);

static void fsm_tick(fsm_t *F);

static void fsm_next_or_done(fsm_t *F);

static void fsm_resume_apply(fsm_t *F, fsm_state_t st, uint8_t idx, bool running,
                             bool stopreq, uint8_t est_age_s);

/* Resume ring */

typedef struct {
    uint8_t magic; uint16_t seq;
    uint8_t yOff, mon, day, dow;
    uint8_t hour, min, sec;
    uint8_t relBits; uint8_t motorBits; uint8_t flags;
    uint8_t f1_state, f1_idx, f1_age;
    uint8_t f2_state, f2_idx, f2_age;
    uint8_t csum;
} __attribute__((packed)) ee_log_t;

static uint16_t ee_log_nextSeq = 0;

static uint8_t ee_log_nextSlot = 0;

static void ee_log_init(void);

static void ee_log_write_snapshot(void);

static bool ee_log_read_slot(uint8_t slot, ee_log_t *rec);

static int8_t ee_log_find_latest_slot(uint16_t *pSeq);

static void ee_log_dump_latest(uint8_t maxLines);

static uint32_t rtc_to_seconds2000(uint16_t y, uint8_t m, uint8_t d, uint8_t hh, uint8_t mm, uint8_t ss);

```

```

static uint32_t seconds2000_now(void);

static uint32_t seconds_diff(uint16_t y, uint8_t m, uint8_t d, uint8_t hh, uint8_t mm, uint8_t ss);


/* Misc */

static inline uint32_t millis(void);

static inline bool elapsed(uint32_t start, uint32_t dur);


/* SINGLE-DRAIN task */

static void drain_single_task(void);


/* ===== LCD INTERNALS ===== */

#define LCD_BL 0x08
#define LCD_EN 0x04
#define LCD_RW 0x02
#define LCD_RS 0x01

static uint8_t lcd_backpack = LCD_BL;


static void lcd_send(uint8_t val, bool rs);

static void lcd_cmd(uint8_t cmd);

static void lcd_char(char c);

static void pcf8574_write(uint8_t data);


static char lcd_line1[64] = {0};
static char lcd_line2[64] = {0};

static uint8_t l1_off = 0, l2_off = 0;

static uint32_t lScrollMark = 0;


/* ===== INTERRUPTS ===== */

void __interrupt(high_priority) isr(void) {

```

```

if (INTCONbits.TMR0IF) {
    INTCONbits.TMR0IF = 0;

    TMR0H = (uint8_t)((65536U - 500U) >> 8);
    TMR0L = (uint8_t)((65536U - 500U) & 0xFF);

    g_millis++;
}

if (PIR1bits.RC1IF) {
    if (RCSTA1bits.OERR) { RCSTA1bits.CREN = 0; RCSTA1bits.CREN = 1; }
    if (RCSTA1bits.FERR) { volatile char dump = RCREG1; (void)dump; }
    else {
        char c = RCREG1;

        uint8_t i = rxHead % UART_RX_BUF_SZ;
        rxBuf[i] = c;

        rxHead = (i + 1) % UART_RX_BUF_SZ;
    }
}

}

/* ===== UTIL ===== */
static inline uint32_t millis(void) {
    uint32_t t;

    uint8_t gieh = INTCONbits.GIEH, giel = INTCONbits.GIEL;
    INTCONbits.GIEH = 0; INTCONbits.GIEL = 0;

    t = g_millis;

    INTCONbits.GIEH = gieh; INTCONbits.GIEL = giel;

    return t;
}

static inline bool elapsed(uint32_t start, uint32_t dur) { return (uint32_t)(millis() - start) >= dur; }

```

```

/* ===== MAIN ===== */

int main(void) {

    init_clock();

    init_gpio();

    init_timer0();

    init_uart();

    init_i2c();


    bool rtc_osf = (rtc_read_reg(0x0F) & 0x80u) != 0;
    rtc_sanitize();


    lcd_init();
    load_settings();


    if (rtc_osf) {
        uint16_t y; uint8_t mo,dt,dw,hh,mm,ss;
        if (rtc_shadow_load(&y,&mo,&dt,&dw,&hh,&mm,&ss)) {
            rtc_write_full(ss,mm,hh,dw,dt,mo,y);
            uint8_t st = rtc_read_reg(0x0F); if (st & 0x80u) { st &= (uint8_t)~0x80u; rtc_write_reg(0x0F, st); }
        }
    }


    /* Multi-line banner/help */
    print_help();


    F1 = (fsm_t){ .state=ST_IDLE, .sequence={1,2,0,0}, .idx=0, .motor=1, .tMark=0, .running=false,
.stop_requested=false, .loop=true };

    F2 = (fsm_t){ .state=ST_IDLE, .sequence={3,4,5,0}, .idx=0, .motor=2, .tMark=0, .running=false,
.stop_requested=false, .loop=true };

```

```

strcpy(lcd_line1, "RealTech Systems - LLS Test Bench");

lcd_line2[0] = 0;


ee_log_init();

uint16_t lastSeq=0; int8_t lastSlot = ee_log_find_latest_slot(&lastSeq);

if (lastSlot >= 0) {
    ee_log_t rec; if (ee_log_read_slot((uint8_t)lastSlot, &rec)) {
        uint32_t delta_s = seconds_diff(2000+rec.yOff, rec.mon, rec.day, rec.hour, rec.min, rec.sec);
        uint8_t f1_age_now = (rec.f1_age + (delta_s > 255 ? 255 : (uint8_t)delta_s));
        uint8_t f2_age_now = (rec.f2_age + (delta_s > 255 ? 255 : (uint8_t)delta_s));

        R1_LAT = (rec.relBits & 0x01)?1:0; R2_LAT = (rec.relBits & 0x02)?1:0; R3_LAT = (rec.relBits &
0x04)?1:0;

        R4_LAT = (rec.relBits & 0x08)?1:0; R5_LAT = (rec.relBits & 0x10)?1:0;

        MR1_LAT = (rec.motorBits & 0x01)?1:0; MR2_LAT = (rec.motorBits & 0x02)?1:0;

        bool f1run=(rec.flags&0x01)!=0, f2run=(rec.flags&0x02)!=0; bool f1stop=(rec.flags&0x08)!=0,
f2stop=(rec.flags&0x10)!=0;

        if (f1run) fsm_resume_apply(&F1, (fsm_state_t)rec.f1_state, rec.f1_idx, true, f1stop,
f1_age_now);

        if (f2run) fsm_resume_apply(&F2, (fsm_state_t)rec.f2_state, rec.f2_idx, true, f2stop,
f2_age_now);

        log_event("RESUME");
    }
}


char line[UART_RX_BUF_SZ];

uint8_t last_dow = 0xFF;


while (1) {
    static uint32_t rtcPoll = 0;

```

```
if (elapsed(rtcPoll, 1000)) {
```

```
    rtcPoll = millis();
```

```
    uint8_t s,m,h,dow; rtc_read(&s,&m,&h,&dow);
```

```
    if (dow != last_dow) {
```

```
        last_dow = dow;
```

```
        if (!today_enabled()) {
```

```
            fsm_request_stop(&F1);
```

```
            fsm_request_stop(&F2);
```

```
            ee_log_write_snapshot();
```

```
        }
```

```
    }
```

```
}
```

```
if (uart_line_available(line, sizeof(line))) parse_command(line);
```

```
/* single-drain task ticker */
```

```
drain_single_task();
```

```
fsm_tick(&F1);
```

```
fsm_tick(&F2);
```

```
lcd_task();
```

```
}
```

```
}
```

```
/* ===== INIT ===== */
```

```
static void init_clock(void) {
```

```
    OSCCONbits.IRCF = 0b111; OSCCONbits.SCS = 0b10;
```

```
    ANSELA = 0x00; ANSELB = 0x00; ANSELC = 0x00;
```

```

}

static void init_gpio(void) {
    R1_TRIS=0; R2_TRIS=0; R3_TRIS=0; R4_TRIS=0; R5_TRIS=0; MR1_TRIS=0; MR2_TRIS=0;
    FS1_TRIS=1; FS2_TRIS=1; FS3_TRIS=1; FS4_TRIS=1; FS5_TRIS=1;
    INTCON2bits.RBPU = 0; WPUBbits.WPUB5 = 1;
    relays_all_off();
}

static void init_timer0(void) {
    T0CONbits.T08BIT=0; T0CONbits.T0CS=0; T0CONbits.PSA=0; T0CONbits.T0PS=0b010;
    TMR0H=(uint8_t)((65536U-500U)>>8); TMR0L=(uint8_t)((65536U-500U)&0xFF);
    INTCONbits.TMR0IF=0; INTCONbits.TMR0IE=1; RCONbits.IPEN=1;
    INTCON2bits.TMR0IP=1; /* high-priority TMR0 */
    INTCONbits.GIEL=1; INTCONbits.GIEH=1; T0CONbits.TMR0ON=1;
}

static void init_uart(void) {
    TRISCbits.TRISC6=0; TRISCbits.TRISC7=1;          /* TX out, RX in */
    BAUDCON1bits.BRG16=1; TXSTA1bits.BRGH=1; SPBRGH1=0x01; SPBRG1=0xA0; // 9600
    RCSTA1bits.SPEN=1; TXSTA1bits.TXEN=1; RCSTA1bits.CREN=1;
    IPR1bits.RC1IP=1;                                /* RX high-priority */
    PIR1bits.RC1IF=0; PIE1bits.RC1IE=1;
}

static void init_i2c(void) {
    TRISCbits.TRISC3=1; TRISCbits.TRISC4=1;
    SSP1STAT=0x00; SSP1STATbits.SMP=1;                /* Standard mode */
    SSP1CON1=0b00101000; SSP1ADD=39; SSP1CON1bits.SSPEN=1; // 100kHz @ 16MHz
}

/* ===== I2C LL ===== */

static void i2c_wait(void){ while((SSP1CON2&0x1F) | ((SSP1STAT&0x04))){} }

```

```

static void i2c_start(void){ i2c_wait(); SSP1CON2bits.SEN=1; }

static void i2c_restart(void){ i2c_wait(); SSP1CON2bits.RSEN=1; }

static void i2c_stop(void){ i2c_wait(); SSP1CON2bits.PEN=1; }

static bool i2c_write(uint8_t b){ i2c_wait(); SSP1BUF=b; i2c_wait(); return !SSP1CON2bits.ACKSTAT; }

static uint8_t i2c_read(bool ack){ uint8_t v; i2c_wait(); SSP1CON2bits.RCEN=1; i2c_wait(); v=SSP1BUF;
i2c_wait(); SSP1CON2bits.ACKDT=ack?0:1; SSP1CON2bits.ACKEN=1; return v; }

/* ===== LCD (PCF8574) ===== */

static void pcf8574_write(uint8_t data){
    i2c_start(); i2c_write((I2C_LCD_ADDR<<1)|0); i2c_write(data); i2c_stop();
}

static void lcd_pulse(uint8_t d){ pcf8574_write((uint8_t)(d|LCD_EN)); __delay_us(1);
pcf8574_write((uint8_t)(d&~LCD_EN)); __delay_us(50); }

static void lcd_send4(uint8_t nibble, bool rs){
    uint8_t d = (uint8_t)((uint8_t)((nibble & 0x0F) << 4) | lcd_backpack | (rs ? LCD_RS : 0));
    lcd_pulse(d);
}

static void lcd_send(uint8_t v, bool rs){ lcd_send4((uint8_t)(v>>4),rs); lcd_send4((uint8_t)(v&0x0F),rs); }

static void lcd_cmd(uint8_t cmd){ lcd_send(cmd,false); }

static void lcd_char(char c){ lcd_send((uint8_t)c,true); }

static void lcd_init(void){
    __delay_ms(50);
    lcd_send4(0x03,false); __delay_ms(5);
    lcd_send4(0x03,false); __delay_us(150);
    lcd_send4(0x03,false); lcd_send4(0x02,false);
    lcd_cmd(0x28);
    lcd_cmd(0x08);
    lcd_cmd(0x01); __delay_ms(2);
    lcd_cmd(0x06);

```



```

    lcd_cmd(0x0C);
}

static void lcd_clear(void){ lcd_cmd(0x01); __delay_ms(2); }

static void lcd_set_cursor(uint8_t row, uint8_t col){
    uint8_t addr = (uint8_t)((row ? 0x40 : 0x00) + col); lcd_cmd((uint8_t)(0x80 | addr));
}

static void lcd_print(const char *s){ while(*s) lcd_char(*s++); }

/* ===== RTC (DS3231) ===== */

static uint8_t bcd2bin(uint8_t b){ return (uint8_t)((b>>4)*10 + (b & 0x0F)); }
static uint8_t bin2bcd(uint8_t b){ return (uint8_t)(((b/10)<<4) | (b%10)); }

static uint8_t rtc_read_reg(uint8_t reg){
    i2c_start(); i2c_write((I2C_RTC_ADDR<<1)|0); i2c_write(reg);

    i2c_restart(); i2c_write((I2C_RTC_ADDR<<1)|1);

    uint8_t v = i2c_read(false); i2c_stop(); return v;
}

static void rtc_write_reg(uint8_t reg, uint8_t val){
    i2c_start(); i2c_write((I2C_RTC_ADDR<<1)|0); i2c_write(reg); i2c_write(val); i2c_stop();
}

static void rtc_sanitise(void){
    uint8_t ctl = rtc_read_reg(0x0E);
    ctl &= (uint8_t)~(1u<<7); // EOSC=0
    ctl |= (1u<<2); // INTCN=1
    rtc_write_reg(0x0E, ctl);
    uint8_t sta = rtc_read_reg(0x0F);
    if (sta & (1u<<7)) { sta &= (uint8_t)~(1u<<7); rtc_write_reg(0x0F, sta); }
    uint8_t hr = rtc_read_reg(0x02);
    if (hr & (1u<<6)) { // 12h->24h
        uint8_t hr12 = bcd2bin((uint8_t)(hr & 0x1F)); bool pm = (hr & 0x20) != 0;
    }
}

```

```

    uint8_t hr24 = (uint8_t)((hr12%12) + (pm?12:0));

    rtc_write_reg(0x02, (uint8_t)(bin2bcd(hr24) & 0x3F));
}
}

static void rtc_read(uint8_t *sec, uint8_t *min, uint8_t *hour, uint8_t *dow) {
    i2c_start(); i2c_write((I2C_RTC_ADDR<<1)|0); i2c_write(0x00);

    i2c_restart(); i2c_write((I2C_RTC_ADDR<<1)|1);

    uint8_t s=i2c_read(true), m=i2c_read(true), h=i2c_read(true), dw=i2c_read(false); i2c_stop();

    if(sec)*sec=bcd2bin(s&0x7F); if(min)*min=bcd2bin(m&0x7F); if(hour)*hour=bcd2bin(h&0x3F);
    if(dow)*dow=(dw&0x07);
}

static void rtc_read_full(uint8_t *sec, uint8_t *min, uint8_t *hour, uint8_t *dow,
    uint8_t *date, uint8_t *month, uint16_t *year) {
    i2c_start(); i2c_write((I2C_RTC_ADDR<<1)|0); i2c_write(0x00);

    i2c_restart(); i2c_write((I2C_RTC_ADDR<<1)|1);

    uint8_t s=i2c_read(true), m=i2c_read(true), h=i2c_read(true), dw=i2c_read(true), dt=i2c_read(true),
    mo=i2c_read(true), yr=i2c_read(false); i2c_stop();

    if(sec)*sec=bcd2bin(s&0x7F); if(min)*min=bcd2bin(m&0x7F); if(hour)*hour=bcd2bin(h&0x3F);

    if(dow)*dow=(dw&0x07); if(date)*date=bcd2bin(dt&0x3F); if(month)*month=bcd2bin(mo&0x1F);
    if(year)*year=(uint16_t)(2000+bcd2bin(yr));
}

static void rtc_write_full(uint8_t sec, uint8_t min, uint8_t hour, uint8_t dow,
    uint8_t date, uint8_t month, uint16_t year) {
    uint8_t yr2=(uint8_t)(year%100);

    i2c_start(); i2c_write((I2C_RTC_ADDR<<1)|0); i2c_write(0x00);

    i2c_write(bin2bcd(sec)); i2c_write(bin2bcd(min)); i2c_write((uint8_t)(bin2bcd(hour)&0x3F));

    i2c_write((dow>=1&&dow<=7)?dow:1); i2c_write(bin2bcd(date)); i2c_write(bin2bcd(month));
    i2c_write(bin2bcd(yr2));

    i2c_stop();
}

```

```

static uint8_t dow_from_ymd(uint16_t y, uint8_t m, uint8_t d) {
    static const uint8_t t[] = {0,3,2,5,0,3,5,1,4,6,2,4}; if(m<3) y-=1;
    uint8_t w=(uint8_t)((y+y/4-y/100+y/400+t[m-1]+d)%7); return (uint8_t)(w==0?1:w+1);
}

static const char* dow_str(uint8_t dow) {
    switch(dow){case 1:return "SUN";case 2:return "MON";case 3:return "TUE";case 4:return "WED";case
5:return "THU";case 6:return "FRI";default:return "SAT";}
}

/* ===== EEPROM ===== */
static void eeprom_write_byte(uint16_t addr, uint8_t val){
    EEADR=(uint8_t)(addr&0xFF); EEADRH=(uint8_t)(addr>>8); EEDATA=val;
    EECON1bits.EEPGD=0; EECON1bits.CFGS=0; EECON1bits.WREN=1;
    uint8_t gieh = INTCONbits.GIEH, giel = INTCONbits.GIEL;
    INTCONbits.GIEH=0; INTCONbits.GIEL=0;
    EECON2=0x55; EECON2=0xAA; EECON1bits.WR=1;
    INTCONbits.GIEH=gieh; INTCONbits.GIEL=giel;
    while(EECON1bits.WR){} EECON1bits.WREN=0;
}

static uint8_t eeprom_read_byte(uint16_t addr){
    EEADR=(uint8_t)(addr&0xFF); EEADRH=(uint8_t)(addr>>8);
    EECON1bits.EEPGD=0; EECON1bits.CFGS=0; EECON1bits.RD=1; return EEDATA;
}

static void eeprom_write_block(uint16_t addr,const uint8_t *buf,uint8_t len){ for(uint8_t i=0;i<len;i++)
eeprom_write_byte(addr+i,buf[i]); }

static void eeprom_read_block(uint16_t addr,uint8_t *buf,uint8_t len){ for(uint8_t i=0;i<len;i++)
buf[i]=eeprom_read_byte(addr+i); }

/* ===== SETTINGS ===== */
static void load_settings(void){

```

```

if (eeprom_read_byte(EA_SIGNATURE_ADDR)!=EA_SIGNATURE_VALUE){

    dayMask=0x7F; restDelayMs=REST_DEFAULT_MS;

    preMotorDelayMs = PREPOST_DEFAULT_MS;

    postMotorRelOffMs = PREPOST_DEFAULT_MS;

    drainSingleMs = DRAIN_SINGLE_DEFAULT_MS;


    eeprom_write_byte(EA_SIGNATURE_ADDR,EA_SIGNATURE_VALUE);
    eeprom_write_byte(EA_DAYMASK_ADDR,dayMask);


    uint32_t v=restDelayMs;
    eeprom_write_byte(EA_REST_MS_ADDR+0,(uint8_t)v);
    eeprom_write_byte(EA_REST_MS_ADDR+1,(uint8_t)(v>>8));
    eeprom_write_byte(EA_REST_MS_ADDR+2,(uint8_t)(v>>16));
    eeprom_write_byte(EA_REST_MS_ADDR+3,(uint8_t)(v>>24));


    v = preMotorDelayMs;
    eeprom_write_byte(EA_PRE_MS_ADDR+0,(uint8_t)v);
    eeprom_write_byte(EA_PRE_MS_ADDR+1,(uint8_t)(v>>8));
    eeprom_write_byte(EA_PRE_MS_ADDR+2,(uint8_t)(v>>16));
    eeprom_write_byte(EA_PRE_MS_ADDR+3,(uint8_t)(v>>24));


    v = postMotorRelOffMs;
    eeprom_write_byte(EA_POST_MS_ADDR+0,(uint8_t)v);
    eeprom_write_byte(EA_POST_MS_ADDR+1,(uint8_t)(v>>8));
    eeprom_write_byte(EA_POST_MS_ADDR+2,(uint8_t)(v>>16));
    eeprom_write_byte(EA_POST_MS_ADDR+3,(uint8_t)(v>>24));


    v = drainSingleMs;
    eeprom_write_byte(EA_DRAIN_MS_ADDR+0,(uint8_t)v);

```

```

    eeprom_write_byte(EA_DRAIN_MS_ADDR+1,(uint8_t)(v>>8));
    eeprom_write_byte(EA_DRAIN_MS_ADDR+2,(uint8_t)(v>>16));
    eeprom_write_byte(EA_DRAIN_MS_ADDR+3,(uint8_t)(v>>24));
} else {
    dayMask=eeprom_read_byte(EA_DAYMASK_ADDR);

    uint32_t v=0;
    v|=(uint32_t)eeprom_read_byte(EA_REST_MS_ADDR+0);
    v|=((uint32_t)eeprom_read_byte(EA_REST_MS_ADDR+1))<<8;
    v|=((uint32_t)eeprom_read_byte(EA_REST_MS_ADDR+2))<<16;
    v|=((uint32_t)eeprom_read_byte(EA_REST_MS_ADDR+3))<<24;
    if (v==0 || v>REST_MAX_MS) v=REST_DEFAULT_MS; restDelayMs=v;

    v=0;
    v|=(uint32_t)eeprom_read_byte(EA_PRE_MS_ADDR+0);
    v|=((uint32_t)eeprom_read_byte(EA_PRE_MS_ADDR+1))<<8;
    v|=((uint32_t)eeprom_read_byte(EA_PRE_MS_ADDR+2))<<16;
    v|=((uint32_t)eeprom_read_byte(EA_PRE_MS_ADDR+3))<<24;
    if (v==0 || v>PREPOST_MAX_MS) v=PREPOST_DEFAULT_MS; preMotorDelayMs=v;

    v=0;
    v|=(uint32_t)eeprom_read_byte(EA_POST_MS_ADDR+0);
    v|=((uint32_t)eeprom_read_byte(EA_POST_MS_ADDR+1))<<8;
    v|=((uint32_t)eeprom_read_byte(EA_POST_MS_ADDR+2))<<16;
    v|=((uint32_t)eeprom_read_byte(EA_POST_MS_ADDR+3))<<24;
    if (v==0 || v>PREPOST_MAX_MS) v=PREPOST_DEFAULT_MS; postMotorRelOffMs=v;

    v=0;
    v|=(uint32_t)eeprom_read_byte(EA_DRAIN_MS_ADDR+0);

```

```

    v|=((uint32_t)eeeprom_read_byte(EE_DRAIN_MS_ADDR+1))<<8;
    v|=((uint32_t)eeeprom_read_byte(EE_DRAIN_MS_ADDR+2))<<16;
    v|=((uint32_t)eeeprom_read_byte(EE_DRAIN_MS_ADDR+3))<<24;
    if (v==0 || v>DRAIN_SINGLE_MAX_MS) v=DRAIN_SINGLE_DEFAULT_MS; drainSingleMs=v;
}
}

static void save_settings(void){
    eeeprom_write_byte(EE_DAYMASK_ADDR,dayMask);
    uint32_t v=restDelayMs;
    eeeprom_write_byte(EE_REST_MS_ADDR+0,(uint8_t)v);
    eeeprom_write_byte(EE_REST_MS_ADDR+1,(uint8_t)(v>>8));
    eeeprom_write_byte(EE_REST_MS_ADDR+2,(uint8_t)(v>>16));
    eeeprom_write_byte(EE_REST_MS_ADDR+3,(uint8_t)(v>>24));

    v=preMotorDelayMs;
    eeeprom_write_byte(EE_PRE_MS_ADDR+0,(uint8_t)v);
    eeeprom_write_byte(EE_PRE_MS_ADDR+1,(uint8_t)(v>>8));
    eeeprom_write_byte(EE_PRE_MS_ADDR+2,(uint8_t)(v>>16));
    eeeprom_write_byte(EE_POST_MS_ADDR+3,(uint8_t)(v>>24)); /* unchanged */

    /* correct post write: */
    v=postMotorRelOffMs;
    eeeprom_write_byte(EE_POST_MS_ADDR+0,(uint8_t)v);
    eeeprom_write_byte(EE_POST_MS_ADDR+1,(uint8_t)(v>>8));
    eeeprom_write_byte(EE_POST_MS_ADDR+2,(uint8_t)(v>>16));
    eeeprom_write_byte(EE_POST_MS_ADDR+3,(uint8_t)(v>>24));

    v=drainSingleMs;
    eeeprom_write_byte(EE_DRAIN_MS_ADDR+0,(uint8_t)v);

```

```

    eeprom_write_byte(EE_DRAIN_MS_ADDR+1,(uint8_t)(v>>8));
    eeprom_write_byte(EE_DRAIN_MS_ADDR+2,(uint8_t)(v>>16));
    eeprom_write_byte(EE_DRAIN_MS_ADDR+3,(uint8_t)(v>>24));
}

/* ===== RTC SHADOW ===== */

static void rtc_shadow_save(uint16_t yr, uint8_t mo, uint8_t dt, uint8_t dow, uint8_t hh, uint8_t mm,
uint8_t ss){

    uint8_t rec[9]; rec[0]=EE_RTC_MAGIC; rec[1]=(uint8_t)(yr-2000); rec[2]=mo; rec[3]=dt; rec[4]=dow;
rec[5]=hh; rec[6]=mm; rec[7]=ss;

    uint16_t sum=0; for(uint8_t i=0;i<8;i++) sum+=rec[i]; rec[8]=(uint8_t)sum;

    eeprom_write_block(EE_RTC_SHADOW_ADDR, rec, 9);
}

static bool rtc_shadow_load(uint16_t *yr, uint8_t *mo, uint8_t *dt, uint8_t *dow, uint8_t *hh, uint8_t
*mm, uint8_t *ss){

    uint8_t rec[9]; eeprom_read_block(EE_RTC_SHADOW_ADDR, rec, 9);

    if (rec[0]!=EE_RTC_MAGIC) return false;

    uint16_t sum=0; for(uint8_t i=0;i<8;i++) sum+=rec[i];

    if (((uint8_t)sum)!=rec[8]) return false;

    uint16_t Y=(uint16_t)(2000 + rec[1]); if (Y<2000 || Y>2099) return false;

    *yr=Y; *mo=rec[2]; *dt=rec[3]; *dow=rec[4]; *hh=rec[5]; *mm=rec[6]; *ss=rec[7];

    if (*dow<1 || *dow>7) *dow = dow_from_ymd(*yr,*mo,*dt);

    return true;
}

/* ===== RELAYS ===== */

static void relays_all_off(void){ R1_LAT=R2_LAT=R3_LAT=R4_LAT=R5_LAT=0; MR1_LAT=MR2_LAT=0; }

static void relays_function_off(uint8_t fn){ if(fn==1){R1_LAT=0;R2_LAT=0;MR1_LAT=0;} else
{R3_LAT=0;R4_LAT=0;R5_LAT=0;MR2_LAT=0;} }

static void motor_set(uint8_t motor,bool on){ if(motor==1) MR1_LAT=on?1:0; else MR2_LAT=on?1:0; }

```

```

static void relay_set(uint8_t relay,bool on){ switch(relay){case 1:R1_LAT=on;break;case
2:R2_LAT=on;break;case 3:R3_LAT=on;break;case 4:R4_LAT=on;break;case 5:R5_LAT=on;break;}}

static bool relay_get(uint8_t relay){ switch(relay){case 1:return R1_LAT;case 2:return R2_LAT;case
3:return R3_LAT;case 4:return R4_LAT;case 5:return R5_LAT;} return false; }

static bool float_triggered(uint8_t relay){

    uint8_t pin=0xFF; switch(relay){case 1:pin=!FS1_PORT;break;case 2:pin=!FS2_PORT;break;case
3:pin=!FS3_PORT;break;case 4:pin=!FS4_PORT;break;case 5:pin=!FS5_PORT;break;default:return false;}

    static uint32_t tStart[6]={0}; static bool last[6]={0};

    bool active=(pin!=0);

    if (active!=last[relay]){ last[relay]=active; tStart[relay]=millis(); return false; }

    if (active && elapsed(tStart[relay],FLOAT_DEBOUNCE_MS)) return true;

    return false;

}

```

/* NEW: debounced float read (level, not edge) for LCD display */

```

static bool float_active_stable(uint8_t idx){

    if (idx < 1 || idx > 5) return false;

    bool raw=false;

    switch(idx){

        case 1: raw = (!FS1_PORT)!=0; break;

        case 2: raw = (!FS2_PORT)!=0; break;

        case 3: raw = (!FS3_PORT)!=0; break;

        case 4: raw = (!FS4_PORT)!=0; break;

        case 5: raw = (!FS5_PORT)!=0; break;

    }

    static bool lastRaw[6]={0};

    static bool stable [6]={0};

    static uint32_t changeT[6]={0};

    /* first-time init */

    if (changeT[idx]==0){ lastRaw[idx]=raw; stable[idx]=raw; changeT[idx]=millis(); return stable[idx]; }

```



```

    if (raw != lastRaw[idx]){ lastRaw[idx]=raw; changeT[idx]=millis(); }

    if (elapsed(changeT[idx], FLOAT_DEBOUNCE_MS)){ stable[idx]=lastRaw[idx]; }

    return stable[idx];
}

/* ===== LCD UI ===== */

static void compose_lcd_lines_busy(void){
    char tmp[96]={0}; bool any=false;

    if (relay_get(1)){strcat(tmp, any?" | ":"");strcat(tmp,"T1 Fill/T2 Drain");any=true;}
    if (relay_get(2)){strcat(tmp, any?" | ":"");strcat(tmp,"T2 Fill/T1 Drain");any=true;}
    if (relay_get(3)){strcat(tmp, any?" | ":"");strcat(tmp,"T3 Fill/T5 Drain");any=true;}
    if (relay_get(4)){strcat(tmp, any?" | ":"");strcat(tmp,"T4 Fill/T3 Drain");any=true;}
    if (relay_get(5)){strcat(tmp, any?" | ":"");strcat(tmp,"T5 Fill/T4 Drain");any=true;}

    if (!any) strcpy(tmp,"");

    strncpy(lcd_line1,tmp,sizeof(lcd_line1)-1);

    bool m1=MR1_LAT!=0, m2=MR2_LAT!=0;

    if (m1&& m2) strcpy(lcd_line2,"Motor 1 & 2 ON");
    else if (!m1 && !m2) strcpy(lcd_line2,"Motor 1 & 2 OFF");
    else if (m1) strcpy(lcd_line2,"Motor 1 ON");
    else strcpy(lcd_line2,"Motor 2 ON");
}

static void compose_lcd_lines_idle(void){
    strncpy(lcd_line1,"RealTech Systems - LLS Test Bench",sizeof(lcd_line1)-1);

    uint8_t s,m,h,dow,dt,mo; uint16_t yr; rtc_read_full(&s,&m,&h,&dow,&dt,&mo,&yr);

    char buf[64]; snprintf(buf,sizeof(buf),"%04u-%02u-%02u %s
%02u:%02u:%02u",yr,mo,dt,dow_str(dow),h,m,s);

    strncpy(lcd_line2,buf,sizeof(lcd_line2)-1);
}

```

```

static void lcd_task(void){

    static uint32_t refresh=0;

    if (!elapsed(refresh,150)) return;

    refresh = millis();

    /* While draining: fixed text on LCD with tank mapping */
    if (drain_single_active) {

        strncpy(lcd_line1, "Draining Process", sizeof(lcd_line1)-1);

        const char* tank = "";

        switch (drain_single_relay) {

            case 1: tank = "Tank 2"; break; /* Relay 1 -> Tank 2 */
            case 2: tank = "Tank 1"; break; /* Relay 2 -> Tank 1 */
            case 3: tank = "Tank 5"; break; /* Relay 3 -> Tank 5 */
            case 4: tank = "Tank 3"; break; /* Relay 4 -> Tank 3 */
            case 5: tank = "Tank 4"; break; /* Relay 5 -> Tank 4 */

            default: tank = ""; break;

        }

        strncpy(lcd_line2, tank, sizeof(lcd_line2)-1);

    } else {

        bool fBusy = (F1.running || F2.running);

        bool anyRelay =
        relay_get(1)||relay_get(2)||relay_get(3)||relay_get(4)||relay_get(5)|| (MR1_LAT||MR2_LAT);

        /* During Function 1/2: if any float is HIGH (debounced), show the requested messages */
        if (fBusy) {

            const char* msg = NULL;

            if (float_active_stable(1)) msg = "Tank 1 Full - Tank 2 Low";

            else if (float_active_stable(2)) msg = "Tank 2 Full - Tank 1 Low";

            else if (float_active_stable(3)) msg = "Tank 3 Full - Tank 4 & 5 Low";

```

```

else if (float_active_stable(4)) msg = "Tank 4 Full - Tank 3 & 5 Low";
else if (float_active_stable(5)) msg = "Tank 5 Full - Tank 4 & 5 Low";

if (msg) {
    strncpy(lcd_line1, msg, sizeof(lcd_line1)-1);
    bool m1=MR1_LAT!=0, m2=MR2_LAT!=0;
    if (m1&&m2) strcpy(lcd_line2,"Motor 1 & 2 ON");
    else if (!m1 && !m2) strcpy(lcd_line2,"Motor 1 & 2 OFF");
    else if (m1) strcpy(lcd_line2,"Motor 1 ON");
    else strcpy(lcd_line2,"Motor 2 ON");
} else if (anyRelay) {
    compose_lcd_lines_busy();
} else {
    compose_lcd_lines_idle();
}
} else {
    if (anyRelay) compose_lcd_lines_busy();
    else      compose_lcd_lines_idle();
}
}

uint8_t len1=(uint8_t)strlen(lcd_line1), len2=(uint8_t)strlen(lcd_line2);
if (elapsed(lScrollMark, LCD_SCROLL_PERIOD_MS)) {
    lScrollMark = millis();
    l1_off = (len1>16)? (uint8_t)((l1_off+1)%len1) : 0;
    l2_off = (len2>16)? (uint8_t)((l2_off+1)%len2) : 0;
}

char w1[17]={0}, w2[17]={0};

```

```

    for (uint8_t i=0;i<16;i++){ w1[i]=(l1_off+i<len1)?lcd_line1[l1_off+i]:' ';
w2[i]=(l2_off+i<len2)?lcd_line2[l2_off+i]:' '; }

    lcd_set_cursor(0,0); lcd_print(w1);

    lcd_set_cursor(1,0); lcd_print(w2);
}

/* ===== FSM CORE ===== */
static void fsm_abort(fsm_t *F){
    if (F->motor==1) relays_function_off(1); else relays_function_off(2);
    F->state=ST_IDLE; F->running=false; F->stop_requested=false;
    log_event("abort"); ee_log_write_snapshot();
}

static void fsm_start(fsm_t *F){
    if (!today_enabled()) { return; }
    if (drain_single_active) { return; } /* block starts during single-drain */
    if (F->state != ST_IDLE || F->running || F->stop_requested) { fsm_abort(F); }
    F->running=true; F->stop_requested=false; F->idx=0; F->state=ST_PRE_MOTOR_DELAY;
    uint8_t r = (F->idx < SEQ_MAX) ? F->sequence[F->idx] : 0; relay_set(r,true); F->tMark=millis();
    log_event("start-pre"); ee_log_write_snapshot();
}

static void fsm_request_stop(fsm_t *F){
    if (!F->running && F->state==ST_IDLE) return;
    if (F->stop_requested) return;
    F->stop_requested=true; log_event("stop-req"); ee_log_write_snapshot();
}

static void fsm_next_or_done(fsm_t *F){
    F->idx++;
    bool end = (F->idx >= SEQ_MAX) || (F->sequence[F->idx] == 0);
    if (end){

```

```

    if (F->loop && !F->stop_requested && today_enabled()){
        F->idx=0; F->state=ST_REST_DELAY; F->tMark=millis();
        log_event("loop-rest"); ee_log_write_snapshot();
    } else {
        F->running=false; F->state=ST_IDLE; F->stop_requested=false;
        log_event("done"); ee_log_write_snapshot();
    }
} else {
    F->state=ST_REST_DELAY; F->tMark=millis();
    log_event("rest"); ee_log_write_snapshot();
}
}

static void fsm_tick(fsm_t *F){
    if (!F->running && F->state==ST_IDLE) return;
    uint8_t r = (F->idx < SEQ_MAX) ? F->sequence[F->idx] : 0;
    switch(F->state){
        case ST_PRE_MOTOR_DELAY:
            if (F->stop_requested){
                F->state=ST_POST_MOTOR_OFF_DELAY; F->tMark=millis();
                log_event("stop->post"); ee_log_write_snapshot(); break;
            }
            if (elapsed(F->tMark, preMotorDelayMs)){
                motor_set(F->motor,true); F->state=ST_MOTOR_ON_WAIT_FLOAT; F->tMark=millis();
                log_event("motor-on"); ee_log_write_snapshot();
            }
            break;
        case ST_MOTOR_ON_WAIT_FLOAT:
            if (F->stop_requested){
                motor_set(F->motor,false); F->state=ST_POST_MOTOR_OFF_DELAY; F->tMark=millis();

```

```

        log_event("motor-off"); ee_log_write_snapshot(); break;
    }
    if (float_triggered(r)){
        motor_set(F->motor,false); F->state=ST_POST_MOTOR_OFF_DELAY; F->tMark=millis();
        log_event("float->post"); ee_log_write_snapshot();
    }
    break;
case ST_POST_MOTOR_OFF_DELAY:
    if (elapsed(F->tMark, postMotorRelOffMs)){
        relay_set(r,false);
        log_event("relay-off");
        if (F->stop_requested){ F->state=ST_REST_DELAY; F->tMark=millis(); log_event("stop-rest");
ee_log_write_snapshot(); }
        else { fsm_next_or_done(F); }
    }
    break;
case ST_REST_DELAY:
    if (elapsed(F->tMark, restDelayMs)){
        if (F->stop_requested){ F->running=false; F->state=ST_IDLE; F->stop_requested=false;
log_event("stopped"); ee_log_write_snapshot(); }
        else if (today_enabled()){
            r = (F->idx < SEQ_MAX) ? F->sequence[F->idx] : 0;
            relay_set(r,true); F->state=ST_PRE_MOTOR_DELAY; F->tMark=millis(); log_event("next-pre");
ee_log_write_snapshot();
        }
        else { F->running=false; F->state=ST_IDLE; log_event("idle/dayoff"); ee_log_write_snapshot(); }
    }
    break;
default: break;
}

```

```
}
```

```
/* Resume logic from EEPROM snapshot */
```

```
static void fsm_resume_apply(fsm_t *F, fsm_state_t st, uint8_t idx, bool running, bool stopreq, uint8_t est_age_s){
```

```
    if (idx >= SEQ_MAX || F->sequence[idx] == 0) idx = 0;
```

```
    F->idx = idx; F->running=running; F->stop_requested=stopreq; F->state=st;
```

```
    uint8_t r = (F->idx < SEQ_MAX) ? F->sequence[F->idx] : 0;
```

```
    switch(st){
```

```
        case ST_PRE_MOTOR_DELAY:
```

```
            relay_set(r,true); motor_set(F->motor,false);
```

```
            if (est_age_s >= (preMotorDelayMs/1000)){ motor_set(F->motor,true); F->state=ST_MOTOR_ON_WAIT_FLOAT; F->tMark=millis(); }
```

```
            else { uint32_t rem=preMotorDelayMs-(uint32_t)est_age_s*1000UL; F->tMark=millis()-(preMotorDelayMs-rem); }
```

```
            break;
```

```
        case ST_MOTOR_ON_WAIT_FLOAT:
```

```
            relay_set(r,true); motor_set(F->motor,true); F->tMark=millis(); break;
```

```
        case ST_POST_MOTOR_OFF_DELAY:
```

```
            relay_set(r,true); motor_set(F->motor,false);
```

```
            if (est_age_s >= (postMotorRelOffMs/1000)){ relay_set(r,false); if (F->stop_requested){ F->state=ST_REST_DELAY; F->tMark=millis(); } else { fsm_next_or_done(F); } }
```

```
            else { uint32_t rem=postMotorRelOffMs-(uint32_t)est_age_s*1000UL; F->tMark=millis()-(postMotorRelOffMs-rem); }
```

```
            break;
```

```
        case ST_REST_DELAY:
```

```
            relay_set(r,false); motor_set(F->motor,false);
```

```
            if (est_age_s >= (restDelayMs/1000)){ if (F->stop_requested){ F->running=false; F->state=ST_IDLE; F->stop_requested=false; } else if (today_enabled()){ relay_set(r,true); F->state=ST_PRE_MOTOR_DELAY; F->tMark=millis(); } else { F->running=false; F->state=ST_IDLE; } }
```

```
            else { uint32_t rem=restDelayMs-(uint32_t)est_age_s*1000UL; F->tMark=millis()-(restDelayMs-rem); }
```

```

        break;

    case ST_IDLE: default: F->running=false; F->state=ST_IDLE; F->stop_requested=false; break;
}
}

/* ===== DAY ENABLE ===== */
static bool dow_enabled(uint8_t dow){
    if (dow < 1 || dow > 7) return false;
    uint8_t bit=(uint8_t)(dow-1);
    return (dayMask & (1u<<bit))!=0;
}

static bool today_enabled(void){
    uint8_t s,m,h,dow; rtc_read(&s,&m,&h,&dow);
    return dow_enabled(dow);
}

/* ===== UART, HELP & CMDS ===== */
static void uart_putc(char c){ while(!PIR1bits.TX1IF){} TXREG1=c; }
static void uart_puts(const char *s){ while(*s){ uart_putc(*s++); } }

static void print_help(void){
    uart_puts("\nLLS Ready.\n");
    uart_puts("Cmds:\n");
    uart_puts(" STARTF1\n");
    uart_puts(" STOPF1\n");
    uart_puts(" STARTF2\n");
    uart_puts(" STOPF2\n");
    uart_puts(" STARTBOTH\n");
    uart_puts(" STOPBOTH\n");
}

```



```

uart_puts(" Txx (rest delay in minutes, e.g., T60 = 60min)\n");
uart_puts(" RTxx (pre/post delays in minutes; e.g., RT1 = 1min)\n");
uart_puts(" DTxx (single-drain seconds; e.g., DT30 = 30s, DT120 = 2min)\n");
uart_puts(" DRAIN1..DRAIN5 (single-drain; LCD shows Draining Process + Tank N)\n");
uart_puts(" DAY01ON..DAY07ON / DAY01OFF..DAY07OFF\n");
uart_puts(" RTC? (also: RTC, TIME, TIME?)\n");
uart_puts(" QUIET / VERBOSE\n");
uart_puts(" LOG / HIST / CLEARLOG\n");
uart_puts(" HELP / CMDS / ?\n");
uart_puts("Set:\n");
uart_puts(" SETTIME=HH:MM[:SS]\n");
uart_puts(" SETDATE=YYYY-MM-DD\n");
uart_puts(" SETDATETIME=YYYY-MM-DD,HH:MM:SS\n");
uart_puts(" SETDOW=01..07\n");
}

```

```

/* format ms -> MM:SS string */

```

```

static void fmt_mmss(uint32_t ms, char *out, uint8_t outsz){
    uint32_t total_s = ms/1000UL;
    uint16_t mm = (uint16_t)(total_s/60UL);
    uint8_t ss = (uint8_t)(total_s%60UL);
    snprintf(out, outsz, "%02u:%02u", (unsigned)mm, (unsigned)ss);
}

```

```

static void print_status_now(void){
    uint8_t s,m,h,dw,dt,mo; uint16_t yr;
    rtc_read_full(&s,&m,&h,&dw,&dt,&mo,&yr);

    char line[48];

```

```

snprintf(line,sizeof(line),"%04u-%02u-%02u %s %02u:%02u:%02u\n",
        yr,mo,dt,dow_str(dw),h,m,s);
uart_puts(line);

uart_puts("Function 1 - "); uart_puts(F1.running ? "ON\n" : "OFF\n");
uart_puts("Function 2 - "); uart_puts(F2.running ? "ON\n" : "OFF\n");

uart_puts("Relay 1 - ");  uart_puts(R1_LAT?"ON\n":"OFF\n");
uart_puts("Relay 2 - ");  uart_puts(R2_LAT?"ON\n":"OFF\n");
uart_puts("Relay 3 - ");  uart_puts(R3_LAT?"ON\n":"OFF\n");
uart_puts("Relay 4 - ");  uart_puts(R4_LAT?"ON\n":"OFF\n");
uart_puts("Relay 5 - ");  uart_puts(R5_LAT?"ON\n":"OFF\n");

uart_puts("Motor 1 - ");  uart_puts(MR1_LAT?"ON\n":"OFF\n");
uart_puts("Motor 2 - ");  uart_puts(MR2_LAT?"ON\n":"OFF\n");

uart_puts("DAYEnable - "); uart_puts(today_enabled()?"YES\n":"NO\n");

char b1[8], b2[8], b3[8];
fmt_mmss(preMotorDelayMs, b1, sizeof(b1));
fmt_mmss(postMotorRelOffMs, b2, sizeof(b2));
fmt_mmss(restDelayMs, b3, sizeof(b3));
uart_puts("RT(pre) - "); uart_puts(b1); uart_puts("\n");
uart_puts("RT(post) - "); uart_puts(b2); uart_puts("\n");
uart_puts("T(rest) - "); uart_puts(b3); uart_puts("\n");

uart_puts("\n");
}

```

```

static bool uart_line_available(char *outLine, uint8_t maxLen){
    static uint8_t rd=0;
    while (rd != rxHead) {
        char c = rxBuf[rd]; rd = (rd+1)%UART_RX_BUF_SZ;
        static uint8_t w=0; static char line[UART_RX_BUF_SZ];
        if (c=='\r' || c=='\n') {
            if (w==0) continue; line[w]=0;
            uint8_t n=(w<(maxLen-1))?w:(maxLen-1);
            for(uint8_t i=0;i<n;i++){ char ch=line[i]; if(ch>='a'&&ch<='z') ch-=32; outLine[i]=ch; }
            outLine[n]=0; w=0;
            uint8_t j=0; for(uint8_t i=0;i<n;i++) if(outLine[i]!=' ') outLine[j++]=outLine[i]; outLine[j]=0;
            return true;
        } else { if (w<(UART_RX_BUF_SZ-1)) line[w++]=c; }
    }
    return false;
}

```

```

static bool parse_uint(const char *s, uint16_t *out, uint8_t minDigits, uint8_t maxDigits){
    uint16_t v=0; uint8_t n=0; while(*s && n<maxDigits && *s>='0'&&*s<='9'){ v=(uint16_t)(v*10+(*s-'0')); s++; n++; }
    if (n<minDigits) return false; *out=v; return true;
}

```

```

static bool parse_hms(const char *s, uint8_t *h, uint8_t *m, uint8_t *sc){
    uint16_t H,M,S=0; const char *p=s;
    if(!parse_uint(p,&H,2,2) || p[2]!=':')return false; p+=3;
    if(!parse_uint(p,&M,2,2))return false; p+=2;
    if(*p!=':'){p++; if(!parse_uint(p,&S,2,2))return false;}
    if(H>23 || M>59 || S>59)return false; *h=(uint8_t)H;*m=(uint8_t)M;*sc=(uint8_t)S; return true;
}

```

```

static bool parse_ymd(const char *s, uint16_t *y, uint8_t *m, uint8_t *d){
    uint16_t Y,M,D; const char *p=s;
    if(!parse_uint(p,&Y,4,4) || p[4]!='-')return false; p+=5;
    if(!parse_uint(p,&M,2,2) || p[2]!='-')return false; p+=3;
    if(!parse_uint(p,&D,2,2))return false;
    if(M<1 || M>12 || D<1 || D>31)return false; *y=Y;*m=(uint8_t)M;*d=(uint8_t)D; return true;
}

```

```

static void send_today_off_msg(void){
    uint8_t s,m,h,dow; rtc_read(&s,&m,&h,&dow);
    if (dow<1 || dow>7) { uart_puts("Today is OFF - No Operation and Process\n"); return; }
    char out[64]; snprintf(out,sizeof(out),"Today is OFF (DAY%02uOFF) - No Operation and Process\n",dow);
    uart_puts(out);
}

```

```

static const char* ok = "OK\n";
static const char* err= "ERR\n";

```

```

static bool is_all_digits(const char *p){
    if (!p || !*p) return false;
    for (const char* q=p; *q; ++q){ if(*q<'0' || *q>'9') return false; }
    return true;
}

```

```

static void start_single_drain(uint8_t relay){
    /* abort any running FSMs to avoid conflicts */
    fsm_abort(&F1);
    fsm_abort(&F2);
}

```

```

/* turn motors OFF for drain */
MR1_LAT=0; MR2_LAT=0;

/* stop any previous drain relay */
if (drain_single_active && drain_single_relay>=1 && drain_single_relay<=5) {
    relay_set(drain_single_relay,false);
}

/* set the requested drain */
relay_set(relay,true);
drain_single_active = true;
drain_single_relay = relay;
drain_single_mark = millis();

log_event("drain-start");
ee_log_write_snapshot();
}

static void parse_command(const char *cmd){
    char buf[UART_RX_BUF_SZ]; uint8_t n=0;
    for(uint8_t i=0; cmd[i] && i<sizeof(buf)-1; i++) if(cmd[i]!=' ') buf[n++]=cmd[i]; buf[n]=0;

    if (!strcmp(buf,"STARTF1")) { if(!today_enabled()){ send_today_off_msg(); return; }
if(drain_single_active){ uart_puts("DRAIN ACTIVE - Start blocked\n"); return; } fsm_start(&F1);
uart_puts(ok); }

    else if (!strcmp(buf,"STOPF1")) { fsm_request_stop(&F1); uart_puts(ok); }

    else if (!strcmp(buf,"STARTF2")) { if(!today_enabled()){ send_today_off_msg(); return; }
if(drain_single_active){ uart_puts("DRAIN ACTIVE - Start blocked\n"); return; } fsm_start(&F2);
uart_puts(ok); }

```

```

else if (!strcmp(buf,"STOPF2")) { fsm_request_stop(&F2); uart_puts(ok); }

else if (!strcmp(buf,"STARTBOTH")) { if(!today_enabled()){ send_today_off_msg(); return; }
if(drain_single_active){ uart_puts("DRAIN ACTIVE - Start blocked\n"); return; } fsm_start(&F1);
fsm_start(&F2); uart_puts(ok); }

else if (!strcmp(buf,"STOPBOTH")) { fsm_request_stop(&F1); fsm_request_stop(&F2); uart_puts(ok); }


else if (!strcmp(buf,"QUIET")) { btVerbose=false; uart_puts(ok); }

else if (!strcmp(buf,"VERBOSE")) { btVerbose=true; uart_puts(ok); }


/* Txx => rest delay in minutes */
else if (buf[0]=='T' && is_all_digits(buf+1)) {
    uint16_t mins=0;
    if(!parse_uint(buf+1,&mins,1,4)) { uart_puts(err); return; }
    uint32_t ms = (uint32_t)mins * 60000UL;
    if (ms==0 || ms>REST_MAX_MS) { uart_puts(err); return; }
    restDelayMs = ms;
    save_settings();
    ee_log_write_snapshot();
    uart_puts(ok);
}

/* RTxx -> set pre/post delays in minutes (1..10) */
else if (!strncmp(buf,"RT",2)) {
    uint16_t mins=0;
    if(!parse_uint(buf+2,&mins,1,3)) { uart_puts(err); return; }
    uint32_t ms = (uint32_t)mins * 60000UL;
    if (ms==0 || ms>PREPOST_MAX_MS) { uart_puts(err); return; }
    preMotorDelayMs = ms;
    postMotorRelOffMs = ms;

```

```

    save_settings();

    ee_log_write_snapshot();

    uart_puts(ok);
}

/* RTC query */

else if (!strcmp(buf,"RTC?") || !strcmp(buf,"RTC") || !strcmp(buf,"TIME?") || !strcmp(buf,"TIME")) {
    uint8_t s,m,h,dw,dt,mo; uint16_t yr;

    rtc_read_full(&s,&m,&h,&dw,&dt,&mo,&yr);

    char out[48];

    snprintf(out,sizeof(out),"%04u-%02u-%02u %s %02u:%02u:%02u\n",
             yr,mo,dt,dow_str(dw),h,m,s);

    uart_puts(out);
}

else if (!strncmp(buf,"SETTIME=",8)){ uint8_t h,m,s; if(!parse_hms(buf+8,&h,&m,&s)){ uart_puts(err);
return; } uint8_t sc,mi,ho,dw,dt,mo; uint16_t yr; rtc_read_full(&sc,&mi,&ho,&dw,&dt,&mo,&yr);
rtc_write_full(s,m,h,dw,dt,mo,yr); rtc_shadow_save(yr,mo,dt,dw,h,m,s); ee_log_write_snapshot();
uart_puts(ok); }

else if (!strncmp(buf,"SETDATE=",8)){ uint16_t y; uint8_t m; uint8_t d;
if(!parse_ymd(buf+8,&y,&m,&d)){ uart_puts(err); return; } uint8_t sc,mi,ho,dw,dt,mo; uint16_t yr;
rtc_read_full(&sc,&mi,&ho,&dw,&dt,&mo,&yr); uint8_t ndow=dow_from_ymd(y,m,d);
rtc_write_full(sc,mi,ho,ndow,d,m,y); rtc_shadow_save(y,m,d,ndow,ho,mi,sc); if(!today_enabled()){
fsm_request_stop(&F1); fsm_request_stop(&F2); } ee_log_write_snapshot(); uart_puts(ok); }

else if (!strncmp(buf,"SETDATETIME=",12)){ const char *p=buf+12; const char *comma=strchr(p,',');
if(!comma){ uart_puts(err); return; } char left[16]={0}, right[12]={0}; uint8_t ln=(uint8_t)(comma-p);
if(ln>=sizeof(left)){ uart_puts(err); return; } memcpy(left,p,ln); strncpy(right,comma+1,sizeof(right)-1);
uint16_t y; uint8_t m; uint8_t d; uint8_t hh,mm,ss; if(!parse_ymd(left,&y,&m,&d) ||
!parse_hms(right,&hh,&mm,&ss)){ uart_puts(err); return; } uint8_t ndow=dow_from_ymd(y,m,d);
rtc_write_full(ss,mm,hh,ndow,d,m,y); rtc_shadow_save(y,m,d,ndow,hh,mm,ss); if(!today_enabled()){
fsm_request_stop(&F1); fsm_request_stop(&F2); } ee_log_write_snapshot(); uart_puts(ok); }

else if (!strncmp(buf,"SETDOW=",7)){ uint16_t dv; if(!parse_uint(buf+7,&dv,2,2) || dv<1 || dv>7){
uart_puts(err); return; } uint8_t sc,mi,ho,dw,dt,mo; uint16_t yr;
rtc_read_full(&sc,&mi,&ho,&dw,&dt,&mo,&yr); rtc_write_full(sc, mi, ho, (uint8_t)dv, dt, mo, yr);

```

```
rtc_shadow_save(yr, mo, dt, (uint8_t)dv, ho, mi, sc); if(!today_enabled()){ fsm_request_stop(&F1);  
fsm_request_stop(&F2); } ee_log_write_snapshot(); uart_puts(ok); }
```

```
/* help */
```

```
else if (!strcmp(buf,"HELP") || !strcmp(buf,"CMDS") || !strcmp(buf,"?")) { print_help(); }
```

```
/* LOG & HIST */
```

```
else if (!strcmp(buf,"LOG")) { print_status_now(); }
```

```
else if (!strcmp(buf,"HIST")) { ee_log_dump_latest(16); }
```

```
else if (!strcmp(buf,"CLEARLOG")) { ramLogHead=0; ramLogCount=0; uart_puts(ok); }
```

```
/* DAYxxON/OFF */
```

```
else if (!strcmp(buf,"DAY",3)){
```

```
    size_t L = strlen(buf);
```

```
    if (L==7 || L==8){
```

```
        char d1=buf[3], d2=buf[4];
```

```
        if (d1>='0'&& d1<='9'&& d2>='0'&& d2<='9'){
```

```
            uint8_t reqDay=(uint8_t)((d1-'0')*10 + (d2-'0')); // 01..07
```

```
            const char* suf = buf+5; bool set_on=false, set_off=false;
```

```
            if (!strcmp(suf,"ON")) set_on=true;
```

```
            else if (!strcmp(suf,"OFF")) set_off=true;
```

```
            if (reqDay>=1 && reqDay<=7 && (set_on || set_off)){
```

```
                uint8_t bit=(uint8_t)(reqDay-1);
```

```
                if (set_on) dayMask |= (1u<<bit);
```

```
                else    dayMask &= ~(1u<<bit);
```

```
                save_settings();
```

```
                uint8_t s,m,h,dow; rtc_read(&s,&m,&h,&dow);
```

```
                if (set_off && reqDay == dow) {
```



```

        fsm_request_stop(&F1);

        fsm_request_stop(&F2);

        ee_log_write_snapshot();
    }

    uart_puts(ok);
} else uart_puts(err);
} else uart_puts(err);
} else uart_puts(err);
}

/* DTxx: single-drain time seconds */
else if (!strcmp(buf,"DT",2) && is_all_digits(buf+2)) {
    uint16_t secs=0;

    if(!parse_uint(buf+2,&secs,1,4)) { uart_puts(err); return; }

    uint32_t ms = (uint32_t)secs * 1000UL;

    if (ms==0 || ms>DRAIN_SINGLE_MAX_MS) { uart_puts(err); return; }

    drainSingleMs = ms;

    save_settings();

    ee_log_write_snapshot();

    uart_puts(ok);
}

/* DRAIN1..DRAIN5: map tank index -> relay, start draining */
else if (!strcmp(buf,"DRAIN",5) && is_all_digits(buf+5)) {
    uint16_t idx=0;

    if(!parse_uint(buf+5,&idx,1,2)) { uart_puts(err); return; }

    if (idx < 1 || idx > 5) { uart_puts(err); return; }

    uint8_t mapped=0;

    switch(idx){

```

```

        case 1: mapped=2; break; /* Tank 1 uses Relay 2 */
        case 2: mapped=1; break; /* Tank 2 uses Relay 1 */
        case 3: mapped=4; break; /* Tank 3 uses Relay 4 */
        case 4: mapped=5; break; /* Tank 4 uses Relay 5 */
        case 5: mapped=3; break; /* Tank 5 uses Relay 3 */
    }
    start_single_drain(mapped);
    uart_puts(ok);
}

else { uart_puts(err); }
}

/* ===== LOGGING (RAM) ===== */
static void log_event(const char *msg){
    (void)msg;
    uint8_t s,m,h,dw,dt,mo; uint16_t yr; rtc_read_full(&s,&m,&h,&dw,&dt,&mo,&yr);
    ram_rec_t rec;
    rec.yOff=(uint8_t)(yr-2000); rec.mon=mo; rec.day=dt; rec.dow=dw; rec.hour=h; rec.min=m; rec.sec=s;
    rec.relBits=0; if(R1_LAT)rec.relBits|=0x01; if(R2_LAT)rec.relBits|=0x02; if(R3_LAT)rec.relBits|=0x04;
    if(R4_LAT)rec.relBits|=0x08; if(R5_LAT)rec.relBits|=0x10;
    rec.flags=0; if(F1.running)rec.flags|=0x01; if(F2.running)rec.flags|=0x02; if(MR1_LAT)rec.flags|=0x04;
    if(MR2_LAT)rec.flags|=0x08; if(dow_enabled(dw))rec.flags|=0x10;

    if (ramLogCount > 0) {
        uint8_t prevIdx = (uint8_t)((ramLogHead + RAM_LOG_DEPTH - 1) % RAM_LOG_DEPTH);
        ram_rec_t *prev = &ramLog[prevIdx];
        if (prev->flags == rec.flags && prev->relBits == rec.relBits) {
            return; /* identical snapshot -> no new RAM entry */
        }
    }
}

```

```

    }
}

ramLog[ramLogHead]=rec;
ramLogHead=(uint8_t)((ramLogHead+1)%RAM_LOG_DEPTH);
if(ramLogCount<RAM_LOG_DEPTH) ramLogCount++;
}

static void print_onoff(bool v){ uart_puts(v?"ON\n":"OFF\n"); }
static void print_yesno(bool v){ uart_puts(v?"YES\n":"NO\n"); }

static void ram_log_dump(void){
    if (ramLogCount==0){ uart_puts("EMPTY\n"); return; }
    uint8_t start=(uint8_t)((ramLogHead + RAM_LOG_DEPTH - ramLogCount)%RAM_LOG_DEPTH);
    for(uint8_t i=0;i<ramLogCount;i++){
        uint8_t idx=(uint8_t)((start+i)%RAM_LOG_DEPTH);
        ram_rec_t *r = &ramLog[idx];

        char line[48];
        snprintf(line,sizeof(line),"%04u-%02u-%02u %s %02u:%02u:%02u\n",
            2000u+r->yOff, r->mon, r->day, dow_str(r->dow), r->hour, r->min, r->sec);
        uart_puts(line);

        uart_puts("Function 1 - "); print_onoff( (r->flags & 0x01)!=0 );
        uart_puts("Function 2 - "); print_onoff( (r->flags & 0x02)!=0 );

        uart_puts("Relay 1 - "); print_onoff( (r->relBits & 0x01)!=0 );
        uart_puts("Relay 2 - "); print_onoff( (r->relBits & 0x02)!=0 );
        uart_puts("Relay 3 - "); print_onoff( (r->relBits & 0x04)!=0 );
        uart_puts("Relay 4 - "); print_onoff( (r->relBits & 0x08)!=0 );
    }
}

```

```

    uart_puts("Relay 5 - "); print_onoff( (r->relBits & 0x10)!=0 );

    uart_puts("Motor 1 - "); print_onoff( (r->flags & 0x04)!=0 );
    uart_puts("Motor 2 - "); print_onoff( (r->flags & 0x08)!=0 );

    uart_puts("DAYEnable - "); print_yn( (r->flags & 0x10)!=0 );

    uart_puts("\n");
}
}

static void ram_log_print_latest(void){
    if (ramLogCount==0){ uart_puts("EMPTY\n"); return; }
    uint8_t idx=(uint8_t)((ramLogHead + RAM_LOG_DEPTH - 1)%RAM_LOG_DEPTH);
    ram_rec_t *r = &ramLog[idx];
    char line[48];
    snprintf(line,sizeof(line),"%04u-%02u-%02u %s %02u:%02u:%02u\n",
        2000u+r->yOff, r->mon, r->day, dow_str(r->dow), r->hour, r->min, r->sec);
    uart_puts(line);
    uart_puts("Function 1 - "); print_onoff( (r->flags & 0x01)!=0 );
    uart_puts("Function 2 - "); print_onoff( (r->flags & 0x02)!=0 );
    uart_puts("Relay 1 - "); print_onoff( (r->relBits & 0x01)!=0 );
    uart_puts("Relay 2 - "); print_onoff( (r->relBits & 0x02)!=0 );
    uart_puts("Relay 3 - "); print_onoff( (r->relBits & 0x04)!=0 );
    uart_puts("Relay 4 - "); print_onoff( (r->relBits & 0x08)!=0 );
    uart_puts("Relay 5 - "); print_onoff( (r->relBits & 0x10)!=0 );
    uart_puts("Motor 1 - "); print_onoff( (r->flags & 0x04)!=0 );
    uart_puts("Motor 2 - "); print_onoff( (r->flags & 0x08)!=0 );
    uart_puts("DAYEnable - "); print_yn( (r->flags & 0x10)!=0 );
    uart_puts("\n");
}

```

```
}
```

```
/* ===== EEPROM SNAPSHOTS (resume) ===== */
```

```
static uint8_t compute_csum(const ee_log_t *r){
```

```
    const uint8_t *p = (const uint8_t*)r;
```

```
    uint16_t s=0; for(uint8_t i=0;i<EE_LOG_REC_SZ-1;i++) s+=p[i]; return (uint8_t)s;
```

```
}
```

```
static void ee_log_init(void){
```

```
    uint16_t lastSeq=0; int8_t slot=ee_log_find_latest_slot(&lastSeq);
```

```
    if (slot<0){ ee_log_nextSeq=1; ee_log_nextSlot=0; } else { ee_log_nextSeq=(uint16_t)(lastSeq+1);  
ee_log_nextSlot=(uint8_t)((slot+1)%EE_LOG_SLOTS); }
```

```
}
```

```
static void fill_snapshot(ee_log_t *rec){
```

```
    memset(rec,0,sizeof(*rec)); rec->magic=EE_LOG_MAGIC; rec->seq=ee_log_nextSeq;
```

```
    uint8_t s,m,h,dw,dt,mo; uint16_t yr; rtc_read_full(&s,&m,&h,&dw,&dt,&mo,&yr);
```

```
    rec->yOff=(uint8_t)(yr-2000); rec->mon=mo; rec->day=dt; rec->dow=dw; rec->hour=h; rec->min=m;  
rec->sec=s;
```

```
    uint8_t relBits=0; if(R1_LAT)relBits|=0x01; if(R2_LAT)relBits|=0x02; if(R3_LAT)relBits|=0x04;  
if(R4_LAT)relBits|=0x08; if(R5_LAT)relBits|=0x10; rec->relBits=relBits;
```

```
    uint8_t motors=0; if(MR1_LAT)motors|=0x01; if(MR2_LAT)motors|=0x02; rec->motorBits=motors;
```

```
    rec-
```

```
>flags=(F1.running?0x01:0)|(F2.running?0x02:0)|(today_enabled()?0x04:0)|(F1.stop_requested?0x08:0)  
|(F2.stop_requested?0x10:0);
```

```
    rec->f1_state=(uint8_t)F1.state; rec->f1_idx=F1.idx; rec->f1_age=(uint8_t)((((millis()-  
F1.tMark)/1000UL)>255?255:((millis()-F1.tMark)/1000UL));
```

```
    rec->f2_state=(uint8_t)F2.state; rec->f2_idx=F2.idx; rec->f2_age=(uint8_t)((((millis()-  
F2.tMark)/1000UL)>255?255:((millis()-F2.tMark)/1000UL));
```

```
    rec->csum=compute_csum(rec);
```

```
}
```

```
static void ee_log_write_snapshot(void){
```

```
    ee_log_t rec; fill_snapshot(&rec);
```

```

uint16_t addr=EE_LOG_BASE + (uint16_t)ee_log_nextSlot*EE_LOG_REC_SZ;
eeprom_write_block(addr,(const uint8_t*)&rec,EE_LOG_REC_SZ);
ee_log_nextSeq++; ee_log_nextSlot=(uint8_t)((ee_log_nextSlot+1)%EE_LOG_SLOTS);
}

static bool ee_log_read_slot(uint8_t slot, ee_log_t *rec){
    if (slot>=EE_LOG_SLOTS) return false;
    uint16_t addr=EE_LOG_BASE + (uint16_t)slot*EE_LOG_REC_SZ;
    eeprom_read_block(addr,(uint8_t*)rec,EE_LOG_REC_SZ);
    if (rec->magic!=EE_LOG_MAGIC) return false;
    if (compute_csum(rec)!=rec->csum) return false;
    return true;
}

static int8_t ee_log_find_latest_slot(uint16_t *pSeq){
    uint16_t bestSeq=0; int8_t bestSlot=-1;
    for(uint8_t i=0;i<EE_LOG_SLOTS;i++){
        ee_log_t r; if(!ee_log_read_slot(i,&r)) continue;
        if (bestSlot<0 || (uint16_t)(r.seq - bestSeq) < 0x8000u){ bestSeq=r.seq; bestSlot=(int8_t)i; }
    }
    if (pSeq) *pSeq=bestSeq; return bestSlot;
}

static void ee_log_dump_latest(uint8_t maxLines){
    int8_t last=ee_log_find_latest_slot(NULL);
    if (last<0){ uart_puts("EMPTY\n"); return; }
    uint8_t count=0; uint8_t slot=(uint8_t)last;
    while(count<maxLines){
        ee_log_t r; if(!ee_log_read_slot(slot,&r)) break;
        char line[96];
        char rel[24]; snprintf(rel,sizeof(rel),"R1:%s R2:%s R3:%s R4:%s R5:%s",
            (r.relBits&0x01)?"ON":"OFF",(r.relBits&0x02)?"ON":"OFF",(r.relBits&0x04)?"ON":"OFF",

```

```

        (r.relBits&0x08)?"ON":"OFF",(r.relBits&0x10)?"ON":"OFF");

    char mot[16]; snprintf(mot,sizeof(mot),"MR1:%s
MR2:%s",(r.motorBits&0x01)?"ON":"OFF",(r.motorBits&0x02)?"ON":"OFF");

    char flg[24]; snprintf(flg,sizeof(flg),"Day:%s F1:%s
F2:%s",(r.flags&0x04)?"ON":"OFF",(r.flags&0x01)?"ON":"OFF",(r.flags&0x02)?"ON":"OFF");

    snprintf(line,sizeof(line),"%04u-%02u-%02u %s %02u:%02u:%02u | %s | %s | %s\n",
        2000u+r.yOff,r.mon,r.day,dow_str(r.dow),r.hour,r.min,r.sec,rel,mot,flg);

    uart_puts(line);

    if(++count>=maxLines) break;

    slot=(uint8_t)((slot + EE_LOG_SLOTS - 1) % EE_LOG_SLOTS);
}
}

/* ===== TIME UTILS ===== */

static bool is_leap(uint16_t y){ return ((y%4==0)&&((y%100)!=0)) || (y%400==0); }

static uint16_t days_before_month(uint16_t y,uint8_t m){ static const uint16_t
cum[12]={0,31,59,90,120,151,181,212,243,273,304,334}; uint16_t d=cum[m-1]; if(m>2&&is_leap(y))
d+=1; return d; }

static uint32_t rtc_to_seconds2000(uint16_t y,uint8_t m,uint8_t d,uint8_t hh,uint8_t mm,uint8_t ss){
    uint16_t y0=2000; uint32_t days=0; for(uint16_t yr=y0; yr<y; yr++) days+=is_leap(yr)?366:365;
    days+=days_before_month(y,m)+(uint32_t)(d-1); return days*86400UL + (uint32_t)hh*3600UL +
(uint32_t)mm*60UL + ss;
}

static uint32_t seconds2000_now(void){ uint8_t s,m,h,dw,dt,mo; uint16_t yr;
rtc_read_full(&s,&m,&h,&dw,&dt,&mo,&yr); return rtc_to_seconds2000(yr,mo,dt,h,m,s); }

static uint32_t seconds_diff(uint16_t y,uint8_t m,uint8_t d,uint8_t hh,uint8_t mm,uint8_t ss){
    uint32_t then=rtc_to_seconds2000(y,m,d,hh,mm,ss), now=seconds2000_now(); return
(now>=then)?(now-then):0;
}

/* ===== SINGLE-DRAIN TASK ===== */

```

```
static void drain_single_task(void){  
    if (!drain_single_active) return;  
    if (elapsed(drain_single_mark, drainSingleMs)){  
        if (drain_single_relay>=1 && drain_single_relay<=5) {  
            relay_set(drain_single_relay,false);  
        }  
        drain_single_active=false;  
        drain_single_relay=0;  
        log_event("drain-done");  
        ee_log_write_snapshot();  
    }  
}
```